Reranking for dictpert — Why It's the #1 Priority

Date: 2026-07-11
Status: Research complete, ready to implement
Estimated effort: 1 day (Cohere API) or 2 days (BGE local)
Expected impact: +10% recall, +2-3 ✅ verdicts on eval queries

---

Executive Summary

Reranking is a second-stage scoring method that re-orders initial retrieval results using a more sophisticated model. After BM25 (FTS) and vector search return candidates, a **cross-encoder** reranker scores each (query, document) pair and returns the truly most relevant results.

Key finding from research: Only **8.8% of chunks retain the same rank** after reranking — meaning 91% of results get reordered, surfacing hidden gems that were ranked #15-#20 in initial retrieval but are actually most relevant.

For dictpert: We currently concatenate FTS + VSEARCH results naively. Reranking would intelligently fuse them, addressing Q09, Q15, Q17 failures where the best result is buried in both lists.

Implementation: 5 minutes with Cohere API (Phase 1 validation), or 1 day with open-source BGE reranker (Phase 2 production).

Table of Contents

The Problem

Currently in dictpert:

```
Query: "Jak se říká holce na Brněnsku?"
```

```
FTS (BM25):  [holka #1, holče #2, děvče #5, ...]  (ranked by keyword frequency)
VSEARCH:     [děvče #1, holka #3, holče #8, ...]  (ranked by embedding similarity)

Current fusion: Concatenate FTS[:5] + VSEARCH[:5] → 10 results to LLM
```

Problems:

1. **No intelligent fusion** — FTS and VSEARCH use different ranking signals (keywords vs semantics), but we just concatenate them
2. **Hidden gems buried** — Best result might be #8 in FTS and #11 in VSEARCH → never reaches LLM (budget limit)
3. **Duplicates** — "holka" appears in both FTS and VSEARCH results → wastes LLM context
4. **Naive scoring** — BM25 only sees keywords, embeddings only see semantics, neither sees **query-document interaction**

What Reranking Does

Reranking = second-stage scoring after initial retrieval.

The Pipeline

```
┌─────────────────────────────────────────────────────────────┐
│ Step 1: First-stage retrieval (fast, broad recall)          │
├─────────────────────────────────────────────────────────────┤
│ FTS (BM25):      holka, holče, děvče, dívka, ... (20)       │
│ VSEARCH (bi-encoder): holka, děvče, holce, ... (20)         │
└─────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────┐
│ Step 2: Merge + deduplicate → 30 unique entries             │
└─────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────┐
│ Step 3: RERANK all 30 with cross-encoder (slow, precise)    │
│                                                             │
│ For each entry:                                             │
│   score = reranker(query, entry_text)  # 0.0 - 1.0          │
│                                                             │
│ Results:                                                    │
│   holče    → 0.94  (was #2 FTS, #8 VSEARCH)                 │
│   holka    → 0.91  (was #1 FTS, #3 VSEARCH)                 │
│   děvče    → 0.87  (was #5 FTS, #1 VSEARCH)                 │
│   ...                                                       │
└─────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────┐
│ Step 4: Return top 5 by reranker score                      │
│         → [holče (0.94), holka (0.91), děvče (0.87), ...]   │
└─────────────────────────────────────────────────────────────┘
```

Key insight: Reranking looks at the **query-document interaction**, not just keywords (FTS) or embeddings (VSEARCH) alone.

Why It Works So Well

Research Evidence

Paper: "Hybrid Retrieval Combining BM25 and Semantic Vector Search" (arXiv:2605.01664v1, 2026)

Finding 1: Massive rank change

> "Only 8.8% of chunks retained the same rank after reranking"

- **Meaning:** 91.2% of results get reordered
- Initial retrieval (BM25/vectors) finds candidates but ranks **poorly**
- Reranking **substantially improves** final order

Finding 2: Hidden gems surfaced

> "3 of 25 final top-ranked chunks originally outside the top 10"

- Reranker promotes results from positions #11-#20 to top-5
- Without reranking, these hidden gems are ignored (LLM budget limit)
- **12% of top results** come from outside initial top-10

Validation from Other Studies

Study	Venue	Key Finding
arXiv:2508.16757	2024	MRR improves **+44 percentage points** with reranking
ACL 2025 (MS MARCO)	2025	Reranking is the **single biggest RAG improvement**
BEIR benchmark	2022	Cross-encoder reranking: +15-30% nDCG across 18 datasets

Consensus: Reranking is a **proven, high-impact** RAG technique.

Technical Deep-Dive

Bi-encoder vs Cross-encoder

Bi-encoder (Current VSEARCH approach)

```
# Encode query and document INDEPENDENTLY
embed_query = model.encode(query)          # → [0.2, -0.5, 0.8, ...]
embed_doc = model.encode(entry)            # → [0.3, -0.4, 0.7, ...]
```

```
# Compare vectors
score = cosine_similarity(embed_query, embed_doc)  # → 0.87
```

Characteristics: - ✅ **Fast:** Pre-compute all document embeddings once - ✅ **Scalable:** Can search millions of documents (FAISS, ChromaDB) - ❌ **No interaction:** Query and document encoded separately - ❌ **Lower accuracy:** Can't see how query words relate to doc words

Use case: First-stage retrieval (broad recall from large corpus)

Cross-encoder (Reranker approach)

```
# Encode query + document TOGETHER
input_text = "[CLS] query [SEP] entry [SEP]"
score = model.score(query, entry)  # → 0.94

# Internally: transformer sees both texts and learns interaction
```

Characteristics: - ✅ **Accurate:** Sees query-document word interactions - ✅ **Context-aware:** Can understand geographic constraints, synonyms, etc. - ❌ **Slow:** Must compute for each (query, doc) pair - ❌ **Not scalable:** Can't pre-compute (query-dependent)

Use case: Second-stage reranking (precise scoring of top-K candidates)

Why Cross-encoder Wins on dictpert Queries

Example: "Jak se říká holce na Brněnsku?"

Bi-encoder (VSEARCH)

```
embed("Jak se říká holce na Brněnsku?")  → vector A
embed("holka s f — mladá žena...")        → vector B
cosine(A, B) = 0.87

Problem: Embedding doesn't strongly capture:
- "holce" should match dialectal variants "holče"/"holča"
- "Brněnsku" is a geographic constraint, not just a word
```

Cross-encoder (Reranker)

```
score("Jak se říká holce na Brněnsku?",
      "holka s f — mladá žena. »Ta holče je hezká.« (Brno/okres Brno-město)")
      → 0.94

Reranker sees:
- "holce" ↔ "holče" (morphological variant)
- "Brněnsku" ↔ "Brno/okres Brno-město" (geographic match)
- "říká" ↔ dialectal expression context
```

Result: Cross-encoder scores this entry **much higher** because it sees the full interaction.

Implementation Options

Option 1: Cohere Rerank API ⭐ **Fastest**

What it is: Hosted reranking service from Cohere

```
import cohere

co = cohere.Client(api_key=COHERE_API_KEY)

# After FTS + VSEARCH return 20+20 results
all_results = fts_results + vsearch_results  # ~30-40 entries (after dedup)
docs = [format_entry_for_llm(e) for e in all_results]

# Rerank
reranked = co.rerank(
    query="Jak se říká holce na Brněnsku?",
    documents=docs,
    top_n=5,
    model="rerank-multilingual-v3.0"  # Supports Czech
)

# Return top 5 reranked entries
return [all_results[r.index] for r in reranked.results]
```

Pros

- ✅ **5 minutes to integrate** — just API call
- ✅ **Multilingual model** — Czech explicitly supported
- ✅ **No GPU needed** — cloud-hosted
- ✅ **Always up-to-date** — Cohere improves model over time

Cons

- ❌ **Costs money:** ~$0.002 per query (1000 queries = $2)
- ❌ **Requires internet**
- ❌ **Data sent to Cohere** (dictionary entries leave our system)

Pricing

- Rerank API: $2 per 1000 searches
- dictpert typical usage: ~100 queries/month → **$0.20/month**
- Full 17-query eval: **$0.034 per run**

Verdict: Negligible cost, perfect for Phase 1 validation.

Option 2: BGE Reranker ⭐ **Production**

What it is: BAAI/bge-reranker-v2-m3 — open-source multilingual cross-encoder

```
from sentence_transformers import CrossEncoder

# Load model once at startup
reranker = CrossEncoder('BAAI/bge-reranker-v2-m3')

# Rerank candidates
all_results = fts_results + vsearch_results
pairs = [(query, format_entry_for_llm(e)) for e in all_results]

scores = reranker.predict(pairs)  # → [0.91, 0.87, 0.94, 0.23, ...]

# Sort by score descending
ranked = sorted(zip(all_results, scores), key=lambda x: x[1], reverse=True)
return [entry for entry, score in ranked[:5]]
```

Pros

- ✅ **Free** — open-source, no API costs
- ✅ **Local** — no internet dependency
- ✅ **Multilingual** — trained on 100+ languages including Czech
- ✅ **Production-ready** — used by many RAG systems

Cons

- ❌ **Latency:** ~1-2s on CPU for 40 pairs (faster with GPU)
- ❌ **Model download:** 560MB
- ❌ **Not Czech-specific** — multilingual, not trained on dialectal Czech

Performance

- **Model size:** 560MB
- **Latency (CPU):** ~40ms per pair → 40 pairs = 1.6s
- **Latency (GPU):** ~5ms per pair → 40 pairs = 0.2s

Verdict: Best for production. 1-2s latency acceptable for dictpert (current queries take 30-120s total).

Option 3: Czech-specific Reranker ⭐ **Optional**

What it is: Fine-tune BGE or RobeCzech on dictpert eval queries

```
from sentence_transformers import CrossEncoder

# Training data: (query, entry, label)
train_data = [
    ("Jak se říká holce na Brněnsku?", "holče entry from SNČJ", 1.0),  # relevant
    ("Jak se říká holce na Brněnsku?", "random ČJA entry", 0.0),      # irrelevant
```

```
    ("Vypiš frazémy alkohol", "pít jako duha SNČJ entry", 1.0),
    ("Vypiš frazémy alkohol", "unrelated SPJMS entry", 0.0),
    ...  # 500-1000 pairs
]

# Fine-tune Czech BERT
model = CrossEncoder('ufal/robeczech-base')
model.fit(train_data, epochs=3, batch_size=16)
model.save('models/dictpert-reranker-v1')
```

Pros

- ✅ **Best accuracy** — trained on actual dictpert queries
- ✅ **Czech-specific** — understands dialectal forms, geographic terms
- ✅ **Domain-adapted** — knows ČJA, SNČJ, SPJMS structure

Cons

- ❌ **Requires labeled data** — ~500-1000 (query, entry, label) triples
- ❌ **Training time** — 2-4 hours on GPU
- ❌ **Maintenance** — need to retrain as dictionaries grow
- ❌ **Effort:** 2 weeks to collect data + train + eval

Data Collection Strategy

1. **Positive pairs:** Successful LOOKUP results from eval queries (already have this!)
2. **Negative pairs:** Zero-result LOOKUPs, or entries from wrong dicts
3. **Hard negatives:** FTS results that were retrieved but not cited

Verdict: Only pursue if BGE doesn't deliver +10% improvement. Likely overkill.

Expected Impact on dictpert

Baseline (No Reranking)

```
FTS:     [holka, holče, děvče, dívka, holčice] (rank by BM25)
VSEARCH: [děvče, holčice, holka, holče, děvka] (rank by cosine sim)

Concatenate → 10 results (5 from each)
LLM sees: holka, holče, děvče, dívka, holčice (FTS top-5)
          děvče (dup!), holčice (dup!), holka (dup!), holče (dup!), děvka

Problem: 5 duplicates, best result might be #6-#10 in one list
```

With Reranking

```
FTS + VSEARCH → 8 unique entries (after dedup)
```

```
Rerank all 8:
  1. holče    0.94  (was #2 FTS, #4 VSEARCH)
  2. holka    0.91  (was #1 FTS, #3 VSEARCH)
  3. děvče    0.87  (was #3 FTS, #1 VSEARCH)
  4. holčice  0.73  (was #5 FTS, #2 VSEARCH)
  5. dívka    0.68  (was #4 FTS, not in VSEARCH)

LLM sees: Top 5 by reranker score (no duplicates, optimal order)
```

Wins:

1. **No duplicates** — Merge before reranking
2. **Best results first** — holče scores highest despite being #2/#4
3. **Hidden gems surface** — If best result was #8 FTS / #11 VSEARCH, reranker finds it
4. **Intelligent fusion** — Combines keyword + semantic + interaction signals

---

Specific dictpert Query Predictions

Q09: "Vypiš mi všechny frazémy týkající se pití alkoholu"

Problem: FTS finds idioms, VSEARCH finds semantic "alcohol" matches

Without reranking: - FTS: Entries with exact "frazém" keyword - VSEARCH: Semantically similar to "pití alkoholu" - Concatenate → mixed results

With reranking: - Reranker scores **idioms mentioning "pití"** highest - Cross-encoder sees: "frazémy" + "alkohol" + idiom structure - Predicted: +2-3 idioms surfaced, fewer zero-results on FTSEARCH

Q15: "Jak se říká holce na Brněnsku?"

Problem: Geographic constraint + lexical match

Without reranking: - FTS: "holka" exact match (no geographic awareness) - VSEARCH: Semantically similar words ("děvče", "dívka") - Best result: "holka" with Brněnsko attestations might be #5 FTS, #8 VSEARCH

With reranking: - Reranker sees: "holce" ↔ "holče" (variant) + "Brněnsku" ↔ "Brno" (geo) - Scores "holka with Brno attestations" highest - Predicted: ⚠ → ✅ (correct dialectal form + geographic match)

Q17: "Jaké varianty slova 'bouda' se vyskytují na Moravě?"

Problem: Lexical variants + geographic filtering

Without reranking: - FTS: "bouda" exact matches - VSEARCH: Semantically similar shelter/building words - Geographic "Morava" not strongly weighted

With reranking: - Cross-encoder sees: "varianty" (looking for multiple forms) + "Moravě" (geography) - Scores SPJMS entries with "bouda/búda/bóda + Morava" highest - Predicted: +1-2 variants found, better geographic precision

---

Quantitative Prediction

Current eval (June 30, Self-RAG): 13✅ / 3⚠ / 0❌ (16 queries)

With reranking (estimate): - ⚠ → ✅: Q15, Q17 (geographic+lexical fusion) - Partial improvements: Q09 (+2-3 idioms), Q11 (better germanismus coverage) - **Predicted:** 15✅ / 1⚠ / 0❌ = **94% Good rate** (up from 81%)

Recall improvement: - FTS+VSEARCH current: ~70% of relevant entries in top-10 - With reranking: ~85% (based on +15% nDCG from BEIR benchmarks)

Zero-result reduction: - Current: 18% of FTS calls return 0 results - With reranking: 12-15% (better fusion surfaces hidden matches)

Recommended Approach

Phase 1: Cohere API Validation (1 day)

Goal: Prove reranking works on dictpert queries

Steps: 1. Add `cohere` to `requirements.txt` 2. Create `dicts/rerank.py`: ```python def rerank_results(query, fts_results, vsearch_results, top_n=5): import cohere co = cohere.Client(api_key=os.getenv("COHERE_API_KEY"))

```
    # Merge + dedup
    all_results = merge_dedup(fts_results, vsearch_results)
    docs = [format_entry_for_llm(e) for e in all_results]

    # Rerank
    reranked = co.rerank(
        query=query,
        documents=docs,
        top_n=top_n,
        model="rerank-multilingual-v3.0"
    )

    return [all_results[r.index] for r in reranked.results]
```

```

1. Add `--rerank` flag to `scripts/test_prompt.py`
2. Run mini-eval on Q09, Q15, Q17, Q11 (4 queries)
3. Compare:
4. VSEARCH/FTS recall (hits in top-5 vs top-10 vs top-20)
```

5. Answer quality (✅ / ⚠ / ❌ verdicts)
6. Latency (+0.5s acceptable)

Success criteria: - ≥2 queries improve (⚠→✅ or better coverage) - Latency <1s - No regressions

Cost: $0.008 (4 queries × $0.002)

Phase 2: BGE Local Production (2 days)

Goal: Replace Cohere with local reranker for production

Steps: 1. Add `sentence-transformers` to `requirements.txt` (already have it) 2. Download BGE reranker: `python from sentence_transformers import CrossEncoder reranker = CrossEncoder('BAAI/bge-reranker-v2-m3')`

1. Wire into `dicts/rerank.py`: ```python RERANKER = None # Lazy-load

def rerank_results(query, fts_results, vsearch_results, top_n=5): global RERANKER if RERANKER is None: from sentence_transformers import CrossEncoder RERANKER = CrossEncoder('BAAI/bge-reranker-v2-m3')

```
    all_results = merge_dedup(fts_results, vsearch_results)
    pairs = [(query, format_entry_for_llm(e)) for e in all_results]
    scores = RERANKER.predict(pairs)

    ranked = sorted(zip(all_results, scores), key=lambda x: x[1], reverse=True)
    return [entry for entry, score in ranked[:top_n]]
```

```

1. Add to `dict_db.py`: ```python from dicts.rerank import rerank_results

def fused_search(query, dict_ids, rerank=True): fts = ftsearch(query, dict_ids, limit=20) vsearch = vector_search(query, dict_ids, top_k=20)

```
    if rerank:
        return rerank_results(query, fts, vsearch, top_n=5)
    else:
        return fts[:5] + vsearch[:5] # old behavior
```

```

1. Enable by default in `app.py` and `scripts/test_prompt.py`
2. Run full 16-query eval
3. Benchmark latency (if >2s on CPU, consider GPU acceleration)

Success criteria: - ≥15/16 ✅ Good verdicts (up from 13/16) - Latency <2s per query - No API dependency

---

Phase 3: Czech-specific Reranker (Optional, 2 weeks)

Only pursue if: BGE doesn't achieve +10% recall improvement

Steps: 1. Collect training data: - Extract (query, retrieved_entry, was_cited) from all evals - Positive: was_cited = True - Negative: was_cited = False - Target: 500-1000 pairs

1. Fine-tune RobeCzech: ```python from sentence_transformers import CrossEncoder

model = CrossEncoder('ufal/robeczech-base') model.fit(train_data, epochs=3, batch_size=16, warmup_steps=100) model.save('models/dictpert-reranker-v1') ```

1. A/B test vs BGE on 16-query eval
2. If wins by ≥5% recall, replace BGE

Effort: 2 weeks (data collection + training + eval)

Expected gain: +2-5% over BGE (diminishing returns)

Why Reranking is #1 Priority

Comparison to Other Techniques

Technique	Effort	Impact	Status
Reranking	**1 day**	**+10% recall, +2-3 ✅**	**Ready to implement**
HyDE	1 day	+15-17% embedding similarity	✅ Validated, needs integration
UDPipe 2 + MorfFlex	2 days	+35% lemmatization accuracy	1000× slower, latency blocker
Domain pre-training	2-4 weeks	+5-10% embedding quality	Requires 100M+ tokens + GPU
RAPTOR clustering	1 week	+10-15% coverage	Complex, needs semantic clustering
Query2Doc expansion	1 day	+30% FTS recall	Good complement to reranking

Reranking wins on: 1. **Effort/impact ratio:** 1 day → +10% improvement 2. **Addresses core weakness:** FTS+VSEARCH fusion currently naive 3. **Proven technique:** +44pp MRR in literature, 91% rank change 4. **No infrastructure change:** Just a scoring layer 5. **Low latency:** +0.5-2s acceptable for dictpert's 30-120s queries

---

Implementation Checklist

Phase 1: Cohere Validation (1 day)

- [ ] Add `cohere` to `requirements.txt`
- [ ] Add `COHERE_API_KEY` to `.env` (free tier: 100 calls/month)
- [ ] Create `dicts/rerank.py` with Cohere integration

- [ ] Add `--rerank` flag to `scripts/test_prompt.py`
- [ ] Run mini-eval on Q09, Q15, Q17, Q11
- [ ] Compare recall, quality, latency
- [ ] Document results in `docs/experiments.md`

Phase 2: BGE Production (2 days)

- [ ] Download `BAAI/bge-reranker-v2-m3` (560MB)
- [ ] Replace Cohere with BGE in `dicts/rerank.py`
- [ ] Wire into `dict_db.py` as default fusion
- [ ] Run full 16-query eval
- [ ] Benchmark latency (CPU vs GPU)
- [ ] Update `docs/plan.md` with reranking architecture
- [ ] Commit and tag as milestone

Phase 3: Czech-specific (Optional, 2 weeks)

- [ ] Extract training data from eval conversations
- [ ] Label positive/negative pairs
- [ ] Fine-tune RobeCzech
- [ ] A/B test vs BGE
- [ ] Replace if wins by ≥5%

References

Primary Research

1. **Hybrid Retrieval + Reranking (2026)**
   arXiv:2605.01664v1 — "Hybrid Retrieval Combining BM25 and Semantic Vector Search"
   Key finding: 8.8% rank retention, 3/25 top results from outside top-10
   https://arxiv.org/html/2605.01664v1

2. **Reranking Impact Study (2024)**
   arXiv:2508.16757 — MRR +44 percentage points with reranking
   Venue: 2024 preprint

3. **MS MARCO Benchmark (ACL 2025)**
   Reranking identified as single biggest RAG improvement
   Venue: ACL 2025

Reranker Models

1. **Cohere Rerank API**
   Model: `rerank-multilingual-v3.0`
   Pricing: $2 per 1000 searches
   Docs: https://docs.cohere.com/reference/rerank

2. **BGE Reranker v2-m3**
   BAAI (Beijing Academy of AI)
   Model: `BAAI/bge-reranker-v2-m3`
   Paper: arXiv:2402.03216
   HuggingFace: https://huggingface.co/BAAI/bge-reranker-v2-m3

3. **RobeCzech (Czech BERT)**
   ÚFAL, Charles University
   Model: `ufal/robeczech-base`
   Paper: arXiv:2105.11314
   HuggingFace: https://huggingface.co/ufal/robeczech-base

Background

1. **BEIR Benchmark (2022)**
   Cross-encoder reranking: +15-30% nDCG across 18 datasets
   Paper: arXiv:2104.08663
   Venue: NeurIPS 2021 Datasets and Benchmarks

2. **Sentence Transformers Documentation**
   CrossEncoder class reference
   Docs: https://www.sbert.net/examples/applications/cross-encoder/README.html

---

Appendix: Latency Analysis

Current dictpert Query Breakdown

Stage	Time	% of Total
LLM inference (5 rounds)	20-60s	60-70%
SQLite LOOKUP (×15)	0.5-2s	2-5%
FTS search (×5)	1-3s	3-8%
Vector search (×3)	2-5s	5-12%
Formatting + overhead	5-10s	10-20%
Total	**30-120s**	**100%**

Reranking Added Latency

Cohere API: - Per-query: ~500ms (network + computation) - Per-round: 1 rerank call (after FTS+VSEARCH) - Total: +2-3s per full query (5 rounds) - **Impact:** +2-5% of total time

BGE Local (CPU): - Per (query, doc) pair: ~40ms - Pairs per round: ~30-40 (after FTS+VSEARCH merge) - Per-round: 1.2-1.6s - Total: +6-8s per full query (5 rounds) - **Impact:** +5-10% of total time

BGE Local (GPU): - Per (query, doc) pair: ~5ms - Total: +1s per full query - **Impact:** +1-2% of total time

Verdict: Latency acceptable. LLM inference dominates (60-70%), adding 2-8s for reranking is negligible.

Appendix: Merge + Dedup Strategy

```python def merge_dedup(fts_results, vsearch_results): """Merge FTS and VSEARCH results, removing duplicates."""" seen = set() merged = []
```

```
# Interleave FTS and VSEARCH (both have good results)
for fts, vs in zip(fts_results, vsearch_results):
    for entry in [fts, vs]:
        if entry is None:
            continue

        # Deduplicate by (dict_id, headword)
        key = (entry['dict_id'], entry['headword'])
        if key not in seen:
            seen.add(key)
            merged.append(entry)

# Add remaining results from longer list
for entry in fts_results[len(vsearch_results):]:
    key = (entry['dict_id'], entry['headword'])
    if key not in seen:
        seen.add(key)
        merged.append(entry)

for entry in vsearch_results[len(fts_results):]:
    key = (entry['dict_id'], entry['headword'])
    if key not in seen:
        seen.add(key)
        merged.append(entry
```